

Cheatsheet Hashcat

Comandos	¿Cuál es su función?
<code>hashid '<hash>'</code>	Entrega el tipo de hash que posee el hash copiado. Este puede entregar más de un tipo ordenado de más probable a menos probable
<code>hashid '<hash>' -m</code>	Entrega el tipo de hash que posee el hash copiado junto a su numero de modulo ocupado en la versión de hashcat
<code>hashid hash.txt</code>	Entrega el tipo de hash que posee el hash que esta en el archivo txt. Solo puedes poner un solo hash en el archivo de texto.
<code>hashcat --example-hashes</code>	Nos permite tener un ejemplo de hashes que posee hashcat para evaluar. Podemos comparar con estos ejemplos cuando tenemos dudas de cual podría ser el tipo de hash
<code>hashcat -help grep 'Tipo_hash'</code>	Sirve para saber el modulo del hash cuando sabemos el nombre del hash
<code>hashcat -b -m <modulo_hash></code>	Permite obtener la velocidad con que el hash va a ser crackeado. Sirve para decidir qué tan difícil es el crackeo antes de iniciarlo.
<code>hashcat -O</code>	Incrementa la velocidad del crackeo, pero limita el largo de la contraseña
<code>hashcat -w 3</code>	Incrementa la velocidad del crackeo, pero es recomendado utilizarlo solo cuando el hashcat es el único proceso corriendo en el sistema. Si no lo es, entonces por defecto se utiliza 2, también si se quiere correr en segundo plano se puede utilizar 1.
<code>hashcat -a 0 -m <modulo_hash> hash.txt <lista_diccionario></code>	Realiza un ataque tipo 0, este tipo de ataque es un ataque de diccionario, donde utilizaremos la lista proporcionada para intentar crackear la contraseña
<code>hashcat -a 1 -m <modulo_hash> hash.txt <lista_diccionario1> <lista_diccionario2></code>	Realiza un ataque de tipo 1, este tipo es una ataque de combinación de diccionarios, donde utilizaremos dos listas de diccionarios donde se combinaran de todas las formas posibles para intentar crackear la contraseña

<pre>hashcat -a 3 -m <modulo_hash> hash.txt "<Mascara>"</pre>	Realiza un ataque de tipo 3, este tipo de ataque es un ataque con mascara lo que quiere decir que la contraseña cumple un patrón en particular. Se probará con todas las contraseñas posibles con el patrón proporcionado .
---	---

Para la mascara poseemos los siguientes caracteres que nos permiten representar los siguientes patrones en una mascara:

1. ?l: Letras minúsculas.
2. ?u: Letras mayúsculas.
3. ?d: Número del 0 al 9
4. ?h: 0123456789abcdef
5. ?H: 0123456789ABCDEF
6. ?s: Todos los caracteres especiales
7. ?a: Todos los posibles caracteres

Ejemplo de mascara: “DOMINIO?l?l?l?l?d?d”: Aquí vemos que las contraseñas podrán ser elementos como DOMINIOabcd1900, donde abcd puede ser cualquier letra minúscula y el año 1900 es hasta 1999.

- Importante: Podemos aplicar nuestro propio carácter especial especificando con las banderas -1 hasta -4 donde la sintaxis es: -1 <Posibles caracteres que toma>. Por ejemplo -1 01, con una mascara “DOMINIO?l?l?l?l20?1?d” podría tomar en el año solo valores entre 2000 a 2019. Solo podemos hacer hasta 4 caracteres propios definidos.

<pre>hashcat -a 6 -m <Modulo_hash> hash.txt <Lista_diccionario> <Mascara></pre>	Realiza un ataque de tipo 6, es decir, realiza un ataque hibrido entre una lista de diccionario y una máscara. Por lo que, todos los elementos del diccionario serán finalizados con la máscara correspondiente.
<pre>hashcat -a 7 -m <Modulo_hash> hash.txt <Mascara> <Lista_diccionario></pre>	Realiza un ataque de tipo 6, es decir, realiza un ataque hibrido entre una lista de diccionario y una máscara. Por lo que, todos los elementos del diccionario serán iniciados con la máscara correspondiente.
<pre>crunch <largo_minimo> <largo_maximo> <diccionario> -t <patrón> -o <output_file></pre>	Corresponde a una herramienta que nos permite realizar nuestra propia wordlist entregando los largo mínimo y máximo de

	las palabras, donde el diccionario son las posibles palabras que puede tomar la palabra y el patrón es muy parecido a lo que es la máscara, pero con otros caracteres. Todo esto se guarda en un archivo mencionado con la bandera -o
--	---

Para el patrón de las contraseñas poseemos los siguientes caracteres:

1. @: Letras minúsculas
2. ,: Letras mayúsculas
3. %: Números
4. ^: Símbolos especiales

Ejemplo de patrón: “DOMINIO@@@@19%%”, es el mismo ejemplo anterior pero cambiado por los caracteres que acepta esta herramienta.

<code>python3 cupp.py -i</code>	Es un script de python que nos permite realizar una wordlist sabiendo datos del sistema o persona objetivos. Estos datos pueden ser datos personales como nombres de familiares, mascotas, etc.
<code>hashcat -a 0 -m <Modulo_hash> hash.txt <lista_diccionario> -r rule.txt</code>	Este, aunque en el comando es un ataque tipo diccionario se puede ocupar para cualquier ataque la bandera -r con el archivo rule.txt. El archivo rule.txt contiene ciertas reglas que debe cumplir las palabras del diccionario.
<code>hashcat -a 0 -m <Modulo_hash> hash.txt <lista_diccionario> -g 1000</code>	Realiza lo mismo que el comando explicado anteriormente, pero esta vez las reglas son autogeneradas de forma aleatoria. En este caso, se generaron 1000 reglas aleatorias, mientras más se generen mayor es el rango de acierto, pero mayor es el tiempo de crackeo.

Existen bastantes formas que se pueden aplicar a las reglas pero estas son las principales:

1. l: Convierte todas las letras a minúsculas
2. u: Convierte todas las letras a mayúsculas
3. c: Convierte el primer carácter a mayúscula y los demás en minúscula.
4. C: Convierte el primer carácter en minúscula y los demás en mayúscula.

5. t: Cambia mayúsculas por minúsculas en toda la palabra
6. TN: Cambia mayúsculas por minúsculas solo en el elemento numero N
7. d: Duplica la palabra completa
8. q: Duplica carácter a carácter
9. zN: Duplica el primer carácter
10. ZN: Duplica el último carácter
11. {: Rota toda la palabra un carácter hacia la izquierda
12. }: Rota toda la palabra un carácter hacia la derecha.
13. ^N: Agrega el carácter N al principio de la palabra
14. \$N: Agrega el carácter N al final de la palabra
15. r: Invierte toda la palabra
16. sNX: Cambia el carácter N por el carácter X en toda la palabra

Ejemplos de cada uno:

1. l: Dominio -> dominio
2. u: Dominio -> DOMINIO
3. c: dominio -> Dominio
4. C: Dominio -> dOMINIO
5. t: DoMiNiO -> dOmInIo
6. T2: DoMiNiO -> DOMiNiO
7. d: Dominio -> DominioDominio
8. q: Dominio -> DDoommiinniioo
9. zN: Dominio -> DDominio
10. ZN: Dominio -> Dominioo
11. {: Dominio -> ominioD
12. }: Dominio -> oDomini
13. ^!: Dominio -> !Dominio
14. \$!: Dominio -> Dominio!
15. r: Dominio -> oinimoD
16. so@: Dominio -> D@mini@

Ejemplo concreto: “c so0 sil shH ss5 \$2 \$0 %2 \$6”, Aquí se realiza una regla que pone en mayúscula la primer letra, reemplaza o -> 0, i -> l, h -> H, s -> 5 y agrega 2026 al final de la palabra.